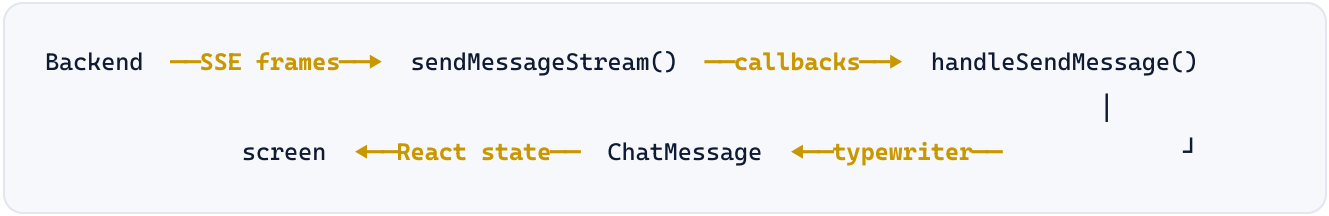
LEARNMATE · ENGINEERING NOTES

How Streaming Text Works

From the backend's Server-Sent Events all the way to the typing effect on screen — with real code from this codebase.

When you send a message, the assistant's reply should appear to **type itself out** word by word, rather than freezing on a loading spinner and then popping in all at once. This document walks through the full path that makes that happen, in four stages:

1. **The backend** pushes the answer as a live stream of small events (SSE).
2. **The API layer** reads that stream and turns each event into a callback.
3. **The page** buffers the incoming text and reveals it gradually (the typewriter).
4. **The message component** renders the growing text with a blinking cursor.



1. What is streaming, really?

A normal HTTP request is *all-or-nothing*: the browser waits, and the entire reply arrives in one lump. That's fine for JSON, but it means the user stares at a spinner until the whole answer is ready.

Server-Sent Events (SSE) keep the connection open and let the server push a sequence of small text *frames* over time. The browser reads them as they arrive. Each frame is just a line that starts with `data:` followed by JSON, and frames are separated by a blank line:

```
data: {"type": "sources", "sources": [ ... ]}

data: {"type": "delta", "text": "Photo"}

data: {"type": "delta", "text": "synthesis is"}

data: {"type": "delta", "text": " the process"}

data: {"type": "done", "response": "Photosynthesis is the process ... ", "notion_urls": []}
```

Our backend endpoint `POST /api/chat/message/stream` emits five kinds of frame:

Frame `type`	Meaning	When
`sources`	The documents used to answer (RAG citations)	Before any text
`delta`	A chunk of the answer text	Repeatedly, as the model generates
`status`	Post-answer work, e.g. "generating notes"	After the text
`done`	The final, authoritative answer + metadata	Once, at the end
`error`	Something failed mid-stream	On failure

> **Key idea.** The `delta` frames are what create the "typing" feeling — each one adds a little more text. The `done` frame carries the complete, final answer so the client never has to trust its own concatenation.

2. Reading the stream in the API layer

We can't use the browser's built-in `EventSource` for SSE here, because it can't attach our `Authorization: Bearer` header. Instead we do a normal `fetch()` and read the response body manually with a **reader**. The bytes arrive in arbitrary pieces, so we buffer them and split on the blank-line frame separator.

`lib/api.ts — sendMessageStream()`

```
export async function sendMessageStream(sessionId, message, files, handlers, signal) {
  const formData = new FormData();
  formData.append('message', message);
  files?.forEach((file) => formData.append('attachments', file));

  const res = await fetch(`${API_BASE}/api/chat/message/stream`, {
    method: 'POST',
    headers: { ...authHeaders() },  // Bearer token — why we can't use EventSource
    body: formData,
    signal,
  });

  if (!res.ok || !res.body) throw new Error(`Stream error ${res.status}`);

  const reader  = res.body.getReader();      // read the body as it arrives
  const decoder = new TextDecoder();
  let buffer   = '';
  let fullText = '';

  while (true) {
    const { done, value } = await reader.read();
    if (done) break;
    buffer += decoder.decode(value, { stream: true });

    const frames = buffer.split('\n\n');   // frames are blank-line separated
    buffer = frames.pop() ?? '';           // keep the trailing partial frame
    for (const frame of frames) handleFrame(frame);
  }
}
```

Each complete frame is parsed and dispatched to the right handler based on its `type`. Notice how `delta` accumulates into `fullText` and reports the *whole answer so far* — that makes the caller's job trivial:

lib/api.ts — handleFrame()

```
const handleFrame = (frame) => {
  const line = frame.split('\n').find((l) => l.startsWith('data: '));
  if (!line) return;
  const evt = JSON.parse(line.slice(6));  // drop "data: "

  switch (evt.type) {
    case 'sources':
      handlers.onSources?.(evt.sources ?? []);
      break;
    case 'delta': {
      fullText += evt.text ?? '';          // accumulate
      handlers.onDelta?.(fullText, evt.text); // report full text so far
      break;
    }
    case 'status': handlers.onStatus?.(evt.status ?? ''); break;
    case 'done':   handlers.onDone?.({ response: evt.response ?? fullText, ... }); break;
    case 'error':  handlers.onError?.(evt.message); break;
  }
};
```

> **Why buffer?** A single `reader.read()` might hand you *half* a frame, or *two-and-a-half* frames. Splitting on `\n\n` and keeping the leftover (`buffer = frames.pop()`) guarantees we only ever parse complete frames.

3. The typewriter — the part that fixes "spinner instead of typing"

Here's the subtle problem. Even with streaming wired up, the reply can still *pop in* instead of typing — because **we don't control how the backend chunks the text**. Some backends send hundreds of tiny `delta` s (looks great). Others send the whole answer in two big chunks, or pack everything into the final `done` frame (looks like a spinner, then a sudden blob).

The robust fix is to **decouple what we receive from what we show**. Incoming text goes into a buffer (`targetText`); a separate animation loop reveals it a few characters at a time (`shown`). No matter how coarsely the server chunks, the screen always types smoothly.

`app/page.tsx — handleSendMessage(): the reveal loop`

```
let targetText  = '';      // full text received so far (from the stream)
let shown       = 0;       // how many characters are currently on screen
let streamEnded = false;   // no more chunks will arrive
let raf         = null;    // the animation-frame handle
let finalPatch  = null;    // final metadata, applied once fully typed

const reveal = () => {
  if (shown < targetText.length) {
    // Ease-out: cover a fraction of the remaining gap each frame (min 2 chars),
    // so big chunks catch up fast but the tail still types character-by-character.
    const remaining = targetText.length - shown;
    shown = Math.min(targetText.length, shown + Math.max(2, Math.ceil(remaining / 22)));
    patchAgent({ content: targetText.slice(0, shown), status: 'chatting' });
    raf = requestAnimationFrame(reveal);
  } else if (!streamEnded) {
    raf = requestAnimationFrame(reveal);  // caught up, but more may come — keep looping
  } else {
    raf = null;                           // fully typed AND stream closed → commit final s
    if (finalPatch) { patchAgent(finalPatch); finalPatch = null; }
  }
};

const startReveal = () => { if (raf === null) raf = requestAnimationFrame(reveal); };
```

The stream handlers now just *feed the buffer* and let the loop do the typing. Notice `onDone` doesn't slam the final text on screen — it stores it in `finalPatch` and lets the reveal loop finish typing first, so the cursor never disappears mid-word:

app/page.tsx — wiring the handlers to the buffer

```
await sendMessageStream(sessionId, userMessage, files, {
  onSources: (sources) => { ensureAgent(); patchAgent({ sources }); },

  onDelta: (fullText) => {
    ensureAgent();
    targetText = fullText;     // feed the buffer …
    startReveal();            //  … and let the loop type it out
  },

  onDone: (evt) => {
    ensureAgent();
    targetText  = evt.response || targetText;
    streamEnded = true;
    finalPatch  = { content: targetText, status: 'completed',   // applied AFTER typing fini
                    notionUrls: evt.notion_urls, sources: evt.sources };
    finalizeSession(sessionId, userMessage);
    startReveal();
  },

  onError: (message) => {
    ensureAgent();
    streamEnded = true;
    finalPatch  = { status: 'completed', error: message };
    startReveal();
  },
});
```

Why `ensureAgent()`?

The empty assistant bubble is created *lazily*, on the first frame that carries content. Until then, `isSending` stays `true` and the loading dots show. The instant real content starts, we swap the loader for the bubble and let it type:

app/page.tsx — ensureAgent() & patchAgent()

```
const ensureAgent = () => {
  if (agentId) return;                          // only once
  const agentMsg = createMessage('', 'agent', 'chatting');
  agentId = agentMsg.id;
  setMessages((prev) => [...prev, agentMsg]);
  setIsSending(false);                         // hide the loading dots
};

const patchAgent = (patch) =>            // update just this one message in React state
  setMessages((prev) => prev.map((m) => (m.id === agentId ? { ...m, ...patch } : m)));
```

> **The tuning knob.** In `Math.ceil(remaining / 22)`, the `22` controls speed. A **bigger** number types **slower**; a **smaller** number types **faster**. Because it reveals a *fraction of the remaining gap*, a full answer always finishes in roughly the same amount of time regardless of length — long answers just move faster.

4. Rendering the growing text

The message component is refreshingly simple — it just renders `message.content` through Markdown and shows a blinking cursor while the status is `'chatting'`. Every `patchAgent()` call updates `content`, React re-renders, and the text visibly grows.

components/chat/chat-message.tsx

```
const isChatting = message.status === 'chatting';

// ... inside the assistant branch ...
<ReactMarkdown ...>{message.content}</ReactMarkdown>

{/* Blinking cursor — only while streaming */}
{isChatting && (
  <span className="inline-block text-accent ml-0.5" style={cursorStyle}>▌</span>
)}
```

The cursor's blink is pure CSS (and it respects "reduced motion" accessibility settings):

```
const cursorStyle = reducedMotion
  ? { opacity: 1 }
  : { animation: 'blink 1s step-end infinite' };
```

Putting it all together

1. User hits send → `setIsSending(true)` → loading dots appear
2. Backend streams `delta` frames → `sendMessageStream` parses them
3. First frame → `ensureAgent()` → dots hidden, empty bubble added
4. Each frame → `targetText` grows → `reveal()` types it out, char by char
5. `done` frame → `finalPatch` stored → applied once typing catches up
6. Status flips to `completed` → cursor disappears → done ✓

The one thing to remember

There are **two independent clocks**: the rate text *arrives* from the network, and the rate it's *revealed* on screen. Keeping them separate is what makes the typing effect smooth and reliable — the UI never depends on the backend chunking its output nicely.

> **Note.** This gives a great typing effect even if the backend sends everything at once. For *true* token-by-token streaming (words appearing exactly as the model generates them), the backend must emit incremental `delta` frames as tokens are produced — that part lives in the server, not the frontend.
